

Algoritmi-Modulo algoritmi per bioinformatica

Quicksort-Capitolo 7 [CLRS]

Roberto Posenato

versione 2026-1, 19/03/2026

1

QUICKSORT

- Idea generale
- $\text{PARTITION}(A, p, r)$
- Analisi di complessità di QUICKSORT

*“Tell me and I forget, teach me and I remember,
involve me and I learn.”*

Benjamin Franklin

*“A student who achieves knowledge through free investigation
and spontaneous effort will be able to retain that knowledge
and will have acquired a methodology that can serve for a li-
fetime.”*

Jean Piaget

Scopo della lezione

- Introdurre l'algoritmo QUICKSORT con l'analisi di complessità nel caso medio.

Nota!

In questa lezione consideriamo il problema dell'ordinamento d'interi senza perdere di generalità.

I risultati valgono per qualsiasi sequenza di valori confrontabili fra loro mediante una relazione d'ordine.

QUICKSORT

Idea generale

È un algoritmo che:

- ordina sul posto (come INSERTIONSORT)
- è basato sulla tecnica del *divide et impera* (come MERGESORT)
- ha complessità $\Theta(n^2)$ nel caso peggiore e $\Theta(n \lg n)$ nel caso medio (qui si dimostra il caso medio con ulteriore assunzione che i valori siano tutti diversi)
- nel caso medio, le costanti moltiplicative nascoste da Θ sono abbastanza piccole. Quindi l'algoritmo risulta essere molto efficiente in pratica ed è tra i più usati per l'ordinamento interno
- In Java viene usato, nella sua versione parallela, quando si devono ordinare ≥ 44 elementi.

QUICKSORT

Algoritmo di tipo *divide et impera*

Divide partiziona (riorganizzando) l'array $A[p : r]$ in due sottoarray $A[p : q - 1]$ e $A[q + 1 : r]$ tali che:

- ogni elemento del primo sottoarray è minore o uguale al **pivot** $A[q]$,
- ogni elemento del secondo sottoarray è maggiore del pivot $A[q]$.

Impera ordina i due sottoarray in modo ricorsivo fino a quando la dimensione dell'array è 0 o 1 (quando $p \geq r$).

Combina dato che i due sottoarray sono ordinati al ritorno della chiamata ricorsiva, non serve alcuna ulteriore fase di combinazione.

QUICKSORT

Algoritmo generale

PARTITION sceglie un valore dell'array (*pivot*), modifica l'array in modo che tutti gli elementi minori o uguali al pivot stiano alla sua sinistra mentre gli altri alla sua destra e restituisce l'indice q del pivot.

Risolve ricorsivamente i due sottoarray creati da PARTITION.

Per ordinare un intero array A , si deve invocare $\text{QUICKSORT}(A, 1, n)$.

Algoritmo: $\text{QUICKSORT}(A, p, r)$

Input: A array di interi**Output:** A array ordinato con $\leq$

```
1: if ( $p < r$ ) then  
2:    $q = \text{PARTITION}(A, p, r)$   
3:    $\text{QUICKSORT}(A, p, q - 1)$   
4:    $\text{QUICKSORT}(A, q + 1, r)$ 
```

QUICKSORT

PARTITION(A, p, r)

Componente fondamentale dell'algoritmo.

Algoritmo: PARTITION(A, p, r)

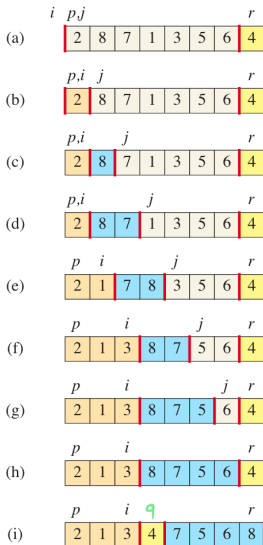
Input: A array di interi

Output: q e A modificato $A[p : q - 1] \leq A[q] < A[q + 1 : r]$

```
1:  $x = A[r]$  // pivot. Scegliere ultimo valore è utile per il for.
2:  $i = p - 1$  // indice ultimo elemento sottoarray sinistro
3: for  $j = p$  to  $r - 1$  do
4:     if ( $A[j] \leq x$ ) then // se l'elemento deve andare nel sottoarray sx
5:          $i = i + 1$  // Posizione per l'elemento da spostare
6:         scambia  $A[i]$  con  $A[j]$ 
7:  $q = i + 1$  // posizione per il pivot
8: scambia  $A[q]$  con  $A[r]$ 
9: return  $q$ 
```

QUICKSORT

PARTITION(A, p, r)



Algoritmo: PARTITION(A, p, r)

Input: A array di interi

Output: q e A modificato $A[p : q - 1] \leq A[q] < A[q + 1 : r]$

1: $x = A[r]$

2: $i = p - 1$

3: **for** $j = p$ **to** $r - 1$ **do**

4: **if** ($A[j] \leq x$) **then**

5: $i = i + 1$

6: scambia $A[i]$ con $A[j]$

7: $q = i + 1$

8: scambia $A[q]$ con $A[r]$

9: **return** q

QUICKSORT

PARTITION(A, p, r): **invariante di ciclo**

Per dimostrare la correttezza di PARTITION(A, p, r), consideriamo la seguente invariante di ciclo:

*All'inizio di ogni iterazione del ciclo **for**, per qualsiasi indice k dell'array, vale almeno una delle seguenti proprietà:*

- ❶ *se $p \leq k \leq i$, allora $A[k] \leq x$;*
 - ❷ *se $i + 1 \leq k \leq j - 1$, allora $A[k] > x$;*
 - ❸ *se $k = r$, allora $A[k] = x$;*
- Da notare che non si considerano i valori con indice k t.c. $j \leq k \leq r - 1$ perché sono valori non ancora *considerati* (e quindi non ancora posizionati correttamente).

QUICKSORT

PARTITION(A, p, r): **invariante di ciclo**

- ✓ **Inizializzazione:** prima della I iterazione, $i = p - 1$ e $j = p$. Non ci sono valori né tra p e $i = p - 1$ né tra $i + 1 = p$ e $j - 1 = p - 1$. Le prime due proprietà dell'invariante sono banalmente soddisfatte. Inoltre, l'istruzione $x = A[r]$ soddisfa la III proprietà dell'invariante.
- ✓ **Conservazione:** Ci sono due possibilità.
 - ❶ Se $A[j] > x$ (linea 4), allora non si fa nulla e $j' = j + 1$: la proprietà (2) dell'invariante è quindi soddisfatta all'inizio del prossimo giro perché $j' - 1 = j$.
 - ❷ Se, invece, $A[j] \leq x$, allora $i' = i + 1$, vengono scambiati $A[i']$ e $A[j]$ e poi $j' = j + 1$. In seguito allo scambio, $A[i'] \leq x$ e la proprietà (1) è soddisfatta. Anche $A[j' - 1] > x$ perché era l'elemento $A[i']$ prima dello scambio. Quindi anche la proprietà (2) è soddisfatta.

QUICKSORT

PARTITION(A, p, r): **invariante di ciclo**

- ✓ **Conclusione:** alla fine del ciclo $j = r$. Ogni posizione dell'array si trova in uno dei tre insiemi descritti nell'invariante.

L'istruzione alla linea 8 posiziona l'elemento pivot nella giusta posizione e, quindi, PARTITION fa esattamente quello che ci si aspetta: riordina l'array rispetto all'elemento pivot.

Complessità computazionale di PARTITION

Il tempo di computazione ha sempre lo stesso ordine di grandezza per qualsiasi input: $\Theta(n)$, dove $n = r - p + 1$.

QUICKSORT

Analisi complessità di QUICKSORT

Il tempo di esecuzione (complessità) dipende, a parità di n , da quanto il partizionamento è bilanciato.

- Se il partizionamento è bilanciato (PARTITION restituisce sempre l'elemento che divide a metà l'array), l'algoritmo ha le stesse prestazioni di MERGESORT: $\Theta(n \lg n)$.

- Se il partizionamento è sbilanciato (PARTITION restituisce sempre un elemento tale che uno dei due sottoarray ha solo pochi elementi, un numero costante), l'algoritmo ha le stesse prestazioni di INSERTIONSORT.

Algoritmo: QUICKSORT(A, p, r)

Input: A array di interi

Output: A array ordinato con $\leq$

1: **if** ($p < r$) **then**

2: $q = \text{PARTITION}(A, p, r)$
3: QUICKSORT($A, p, q - 1$)
4: QUICKSORT($A, q + 1, r$)

QUICKSORT

Analisi complessità di QUICKSORT: caso peggiore elementare

- PARTITION restituisce sempre un sottoarray di dimensione $n - 1$ e uno di dimensione $\emptyset$.
- In questo caso il tempo di esecuzione $T(n)$ può essere espresso come

$$\begin{aligned}T(n) &= T(n - 1) + T(\emptyset) + \Theta(n) \\ &= T(n - 1) + \Theta(n)\end{aligned}$$

dove $\Theta(n)$ è il costo di PARTITION.

- L'equazione è la somma aritmetica dei costi di PARTITION, quindi

$$T(n) = \Theta\left(\frac{n(n+1)}{2}\right) = \Theta(n^2) \text{ (caso peggiore)}$$

- Notare che questo caso si verifica anche quando l'array è completamente ordinato! $\Rightarrow$ QUICKSORT è peggiore di INSERTIONSORT.

QUICKSORT

Analisi complessità di QUICKSORT: caso peggiore formale

L'analisi del caso peggiore deve essere fatta in modo più formale per dimostrare che si può verificare in più casi.

- PARTITION restituisce sempre due sottoarray, uno di dimensione d e l'altro di dimensione $n - d - 1$, dove $0 \leq d \leq n - 1$.
- In questo caso il tempo di esecuzione $T(n)$ può essere espresso come

$$T(n) = \max_{0 \leq d \leq n-1} (T(d) + T(n - d - 1)) + \Theta(n)$$

dove $\Theta(n)$ è il costo di PARTITION.

- Si usa il metodo di sostituzione ipotizzando che $T(n) \leq cn^2$

$$\begin{aligned} T(n) &\leq \max_{0 \leq d \leq n-1} (cd^2 + c(n - d - 1)^2) + \Theta(n) \\ &= c \cdot \max_{0 \leq d \leq n-1} (d^2 + (n - d - 1)^2) + \Theta(n) \end{aligned}$$

QUICKSORT

Analisi complessità di QUICKSORT: caso peggiore

$$\begin{aligned} T(n) &\leq \max_{0 \leq d \leq n-1} (cd^2 + c(n-d-1)^2) + \Theta(n) \\ &= c \cdot \max_{0 \leq d \leq n-1} (d^2 + (n-d-1)^2) + \Theta(n) \end{aligned}$$

- $(d^2 + (n-d-1)^2)$ ha il suo massimo quando $d = 0$ o $d = n-1$.
- $\max_{0 \leq d \leq n-1} (d^2 + (n-d-1)^2) \leq (n-1)^2 = n^2 - 2n + 1$
- $T(n) \leq cn^2 - c(2n-1) + \Theta(n) \leq cn^2$
- Quindi $T(n) = O(n^2)$.
- L'analisi del caso peggiore **semplice** aveva dato che $T(n) = \Theta(n^2) = \Omega(n^2)$.
- Con $T(n) = \Omega(n^2) = O(n^2) \Rightarrow T(n) = \Theta(n^2)$ (caso peggiore).

QUICKSORT

Analisi complessità di QUICKSORT: caso migliore

- PARTITION restituisce sempre due sottoarray con dimensione $\left\lfloor \frac{(n-1)}{2} \right\rfloor$ e $\left\lceil \frac{(n-1)}{2} \right\rceil$ rispettivamente.
- In questo caso il tempo di esecuzione $T(n)$ può essere espresso (trascurando troncamenti e arrotondamenti) come

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

dove $\Theta(n)$ è il costo di PARTITION.

- Equazione già risolta per il MERGESORT.

$$T(n) = \Theta(n \lg n) \text{ (caso migliore)}$$

QUICKSORT

Analisi complessità di QUICKSORT: verso il caso medio

- Nelle analisi precedenti si è sempre supposto che a ogni ricorsione la dimensione dei due sottoarray fosse fissata: $(d, n - d - 1)$ nel caso peggiore, $(\lfloor (n - 1)/2 \rfloor, \lceil (n - 1)/2 \rceil)$ nel caso migliore.
- **Caso proporzionalità fissa** : si supponga che PARTITION restituisca sempre due sottoarray con dimensione $\frac{9}{10}n$ e $\frac{1}{10}n$ rispettivamente*.
- In questo caso il tempo di esecuzione $T(n)$ può essere espresso come

$$T(n) = T\left(\frac{9}{10}n\right) + T\left(\frac{1}{10}n\right) + \Theta(n) = T\left(\frac{9}{10}n\right) + T\left(\frac{1}{10}n\right) + cn$$

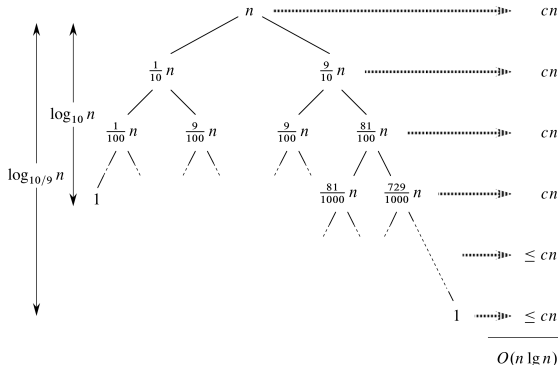
dove $\Theta(n)$ è il costo di PARTITION e c è una costante opportuna.

*sembrerebbe un caso simile al peggiore perché sbilanciato...

QUICKSORT

Analisi complessità di QUICKSORT: verso il caso medio

- Ogni livello ha costo cn finché il ramo $T(\frac{n}{10})$ non diventa 1: $\log n = \Theta(\lg n)$
- Dopo il livello $\log n$, ogni livello costa $< cn$.
- La ricorsione termina alla profondità $\log_{\frac{10}{9}} n = \Theta(\lg n)$
- Costo totale: $O(n \lg n)$.



QUICKSORT

Analisi complessità di QUICKSORT: verso il caso medio

- In generale, non si può supporre che un input sia tale per cui **tutte** le chiamate ricorsive saranno o “caso migliore” o “caso peggiore” o caso con “proporzionalità fissa”.
- *Assumendo che gli input siano distribuiti in modo equiprobabile*, succederà che durante una computazione, delle ricorsioni divideranno la sottoistanza nel modo migliore, altre no.
- Si definisca l'elemento pivot $A[q]$ trovato da PARTITION **buono** se $A[q]$ risiede tra il 25° e il 75° percentile[†].

[†]Il j -percentile è il minimo valore sotto al quale ricade il $j\%$ degli elementi considerati.

QUICKSORT

Analisi complessità di QUICKSORT: verso il caso medio

- $A[q]$ è **buono** $\Rightarrow |A[p : q - 1]|, |A[q + 1 : r]| \leq \frac{3}{4}|A|$.
- 50% di A sono elementi **buoni**.
- Sia X_q la variabile casuale indicatrice: $A[q]$ è *un elemento buono*.
- $\Pr\{X_q = 1\} = E[X_q] = \frac{1}{2}$
- Dati due pivot $A[q]$ e $A[q']$ (uno dopo l'altro), il valore atteso di pivot buoni nella sequenza è $E[X_q + X_{q'}] = E[X_q] + E[X_{q'}] = \frac{1}{2} + \frac{1}{2} = 1$

QUICKSORT

Analisi complessità di QUICKSORT: verso il caso medio

- Quindi, mediamente ogni 2 scelte casuali del pivot si ottiene un valore **buono**.
- Due livelli consecutivi della ricorsione costano complessivamente $O(n)$.
- Dopo una partizione **buona**, la dimensione del sottoproblema più grande è al più $\frac{3}{4}n$.
- Quindi, raggruppando i livelli due a due, la ricorrenza si comporta come

$$T(n) \leq T\left(\frac{3}{4}n\right) + T\left(\frac{1}{4}n\right) + O(n)$$

- La complessità è dello stesso ordine del caso di “proporzionalità fissa”: $T(n) = O(n \lg n)$.

QUICKSORT

QUICKSORT randomizzato

- Assumere che l'input abbia gli elementi distribuiti in modo casuale uniforme è troppo limitante.
- È preferibile mescolare l'input usando un generatore di numeri casuali con distribuzione uniforme per *rendere casuale l'ordine degli elementi*.
- O, in modo equivalente, scegliere l'elemento pivot casualmente e metterlo nella posizione r prima di invocare PARTITION.

Algoritmo: RANDQUICKSORT(A, p, r)

Input: A array di interi

Output: A array ordinato con $\leq$

```
1: if ( $p < r$ ) then
2:    $q = \text{RANDPARTITION}(A, p, r)$ 
3:   RANDQUICKSORT( $A, p, q - 1$ )
4:   RANDQUICKSORT( $A, q + 1, r$ )
```

Algoritmo: RANDPARTITION(A, p, r)

Input: A array di interi

Output: q e A modificato

$A[p : q - 1] \leq A[q] <$
 $A[q + 1 : r]$

1: $i = \text{RANDOM}(p, r)$

2: **scambia** $A[r]$ con $A[i]$

3: **return** PARTITION(A, p, r)

QUICKSORT

Analisi complessità caso medio di QUICKSORT

- RANDQUICKSORT garantisce una scelta uniforme del pivot a ogni chiamata ricorsiva.
- In [CLRS] c'è un'analisi formale e **rigorosa** della complessità nel caso medio.
- L'idea si basa sul fatto che il costo reale dell'algoritmo è *proporzionale* al numero **totale** di confronti (istruzione 4) eseguiti dalla procedura PARTITION
- Si deve quindi determinare il numero **totale** di confronti fatti da PARTITION durante tutta una computazione di QUICKSORT.

QUICKSORT

Analisi complessità caso medio di QUICKSORT

- Data A , si rinominano tutti gli elementi come $z_1, z_2, \dots, z_n$ dove z_i è l' i -esimo elemento più piccolo di A .
- QUICKSORT confronta (in PARTITION) una coppia (z_i, z_j) al più **una** volta in tutta la sua esecuzione.
 - Se uno dei due diventa pivot, può essere confrontato con l'altro.
 - Un elemento pivot non viene poi più confrontato con elementi appartenenti a chiamate ricorsive successive.
- Quindi si definisce la variabile casuale indicatrice

$$X_{ij} = I\{z_i \text{ è confrontato con } z_j\}$$

QUICKSORT

Analisi complessità caso medio di QUICKSORT

- Numero totale di confronti

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}$$

- Numero totale atteso di confronti

$$\begin{aligned} E[X] &= E \left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij} \right] \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}] \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \Pr\{z_i \text{ è confrontato con } z_j\} \end{aligned}$$

- Si deve calcolare $\Pr\{z_i \text{ è confrontato con } z_j\}$

QUICKSORT

Analisi complessità caso medio di QUICKSORT

- Si deve calcolare $\Pr\{z_i \text{ è confrontato con } z_j\}$
- Se si ha un insieme $Z_{ij} = \{z_i, \dots, z_j\}$ e PARTITION sceglie un elemento x tale che $z_i < x < z_j$, z_i e z_j non verranno mai confrontati perché andranno su i due sottoarray distinti.
- Se $x = z_i$ o $x = z_j$, allora z_i e z_j verranno confrontati una volta.
- Gli elementi di Z_{ij} sono equi-probabili. La probabilità di scegliere un elemento particolare è quindi $1/(j - i + 1)$ quando gli elementi sono distinti.

$$\begin{aligned}\Pr\{z_i \text{ è confrontato con } z_j\} &= \Pr\{z_i \text{ o } z_j \text{ è il primo pivot di } Z_{ij}\} \\ &= \Pr\{z_i \text{ è il primo pivot di } Z_{ij}\} \\ &\quad + \Pr\{z_j \text{ è il primo pivot di } Z_{ij}\} \\ &= \frac{2}{j - i + 1}\end{aligned}$$

- (si somma perché sono due eventi mutuamente esclusivi)

QUICKSORT

Analisi complessità caso medio di QUICKSORT

- Ricapitolando

$$\begin{aligned} E[X] &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \Pr\{z_i \text{ è confrontato con } z_j\} \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} \\ &= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} \text{ (cambio variabile } k = j - i) \\ &< \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} \\ &= \sum_{i=1}^{n-1} O(\lg n) \text{ (stima superiore serie armonica)} \\ &= O(n \lg n) \end{aligned}$$

In questa lezione si è visto:

- l'algoritmo QUICKSORT
- l'analisi del caso medio di QUICKSORT.

- Studiare cap. 7 [CLRS].
- Risolvere gli esercizi 7.1-1, 7.1-2, 7.1-4, 7.2-1, 7.2-2, 7.2-3, 7.4-1, 7.4-3, 7.4-5 [CLRS].